```python
# Report valid JSON count and identical-value count per temperature
# ============================================================

def analyze_consistency(results, label):
    valid = [r for r in results if r is not None]
    n_valid = len(valid)

    # Serialize each valid result with sorted keys so identical dicts produce identi
    serialized = [json.dumps(r, sort_keys=True) for r in valid]
    unique_strings = set(serialized)
    n_identical = max(serialized.count(s) for s in unique_strings) if serialized els

    print(f"--- {label} ---")
    print(f"Valid JSON: {n_valid}/5")
    print(f"Unique outputs: {len(unique_strings)}")
    print(f"Largest group of identical outputs: {n_identical}/5")
    print()

analyze_consistency(temp_0_results, "Temperature = 0.0")
analyze_consistency(temp_1_results, "Te  ature = 1.0")
```

Write a message...

+

Sonnet 5 Mediu

Authenticati
You can try a

```python
SUMMARY_PROMPT_V1 = "Summarize this:"

def summarize_v1(letter_text):
    prompt = f"{SUMMARY_PROMPT_V1}\n\n{letter_text}"
    answer, usage = ask_llm(prompt)  # adjust if ask_llm returns just a string
    return answer

print("=== V1 — L002 ===")
print(summarize_v1(L002))

print("\n=== V1 — L006 ===")
print(summarize_v1(L006))
```

python

```python
# ============================================================
# V2 — role, constraints, structured template
# ============================================================
SUMMARY_SYSTEM_PROMPT_V2 = (
    "You are an assistant to a microfinance loan officer. "
```

Write a message...

\+

Sonnet 5 Medium

```python
def summarize_v2(letter_text):
    user_prompt = f"Summarize this loan application:\n\n{letter_text}"
    answer, usage = ask_llm(
        user_prompt,
        system_prompt=SUMMARY_SYSTEM_PROMPT_V2,
        temperature=0.0
    )
    return answer

print("=== V2 — L002 ===")
print(summarize_v2(L002))

print("\n=== V2 — L006 ===")
print(summarize_v2(L006))
```

python

```python
# ============================================================
# Side-by-side comparison
#
```

Write a message...

+ Sonnet 5 Medium

```python
# ----------------------------------------------------------------

def extract_fields(letter_text):
    prompt = build_extract_prompt(letter_text)
    raw = ask_llm(prompt, system_prompt=EXTRACT_SYSTEM_PROMPT, temperature=0)

    # Strip ```json ... ``` or ``` ... ``` fences if the model adds them anyway
    cleaned = raw.strip()
    if cleaned.startswith("```"):
        cleaned = cleaned.split("```")[1]
        if cleaned.startswith("json"):
            cleaned = cleaned[4:]
        cleaned = cleaned.strip()

    try:
        result = json.loads(cleaned)
        return result
    except json.JSONDecodeError as e:
        print(f"⚠️ Failed to parse JSON. Error: {e}")
        print(f"Raw output was:\n{raw}")
        return None
```

Write a message...

+

Sonnet 5 Medium

```python
# ============================================================
# Run on all six letters, collect into a DataFrame
# ============================================================

rows = []
for letter_id, text in LETTERS.items():
    result = extract_fields(text)
    if result is not None:
        result["letter_id"] = letter_id
        rows.append(result)
    else:
        rows.append({"letter_id": letter_id, "applicant_name": None, "amount_ghs": N
                     "purpose": None, "monthly_profit_ghs": None,
                     "has_collateral_or_guarantor": None, "repayment_months": None})

df = pd.DataFrame(rows)
df = df[["letter_id", "applicant_name", "amount_ghs", "purpose",
         "monthly_profit_ghs", "has_collateral_or_guarantor", "repayment_months"]]
df
```

Write a message...

+ Sonnet 5 Medium

```python
def build_brief_prompt(letter_text, extracted_json):
    return (
        f"Loan application letter:\n\"{letter_text}\"\n\n"
        f"Extracted data:\n{json.dumps(extracted_json, indent=2)}\n\n"
        f"Produce the four-section brief."
    )

def generate_brief(letter_text, extracted_json):
    prompt = build_brief_prompt(letter_text, extracted_json)
    return ask_llm(prompt, system_prompt=BRIEF_SYSTEM_PROMPT, temperature=0)


# ============================================================
# Generate briefs for all six letters
# ============================================================

briefs = {}
for letter_id, text in LETTERS.items():
    extracted = extract_fields(text)  # reuse Stage 2 extraction
    briefs[letter_id] = generate_brief(text, extracted)
```